

Razvoj web aplikacija - Predavanje 0

O predmetu

mag.ing. Josip Torić (SIT UNIZD)

05.10.2026.



Studij informacijske tehnologije
Sveučilišta u Zadru

Plan predavanja

- 1 Što je kolegij i koji je cilj
- 2 Što gradimo: sustav prijava na powerlifting natjecanje
- 3 Organizacija nastave, pravila, bodovanje i obrane
- 4 Tehnologije, arhitektura i razvojni workflow
- 5 Kako učiti ovaj kolegij

Ne radimo „tutorial app”. Radimo **inženjerski** projekt: pravila, sigurnost, testovi, deploy.

Cilj kolegija

- Izraditi **full-stack** web aplikaciju: backend + frontend + baza
- Razumjeti **arhitekturu** web sustava i trade-offe
- Ovladati **API dizajnom** i modeliranjem podataka
- Implementirati **autentikaciju** (JWT) i **autorizaciju** (role/ownership)
- Izgraditi SPA frontend: **Vue 3**, routing, state management, UX discipline
- Uspostaviti **workflow**: Git, commitovi, testiranje, CI i deploy

Ishod: student može samostalno izraditi, testirati i deployati manju web aplikaciju.

Što očekujem od vas

- Redovito praćenje predavanja i razumijevanje koncepata, ne samo kopiranje
- Rad u repozitoriju: **commit discipline** i čitljiv kod
- Postavljanje pitanja i rješavanje problema - debugging kao vještina
- Minimalna higijena: formatiranje, error handling, logika na pravom mjestu

Najveća razlika između prolaza i pada je: **razumijete li zašto kod radi.**

Projekt: Sustav prijava na powerlifting natjecanje

Uloge

- **Admin:** kreira natjecanje, kreira klubove i login podatke, vidi sve prijave
- **Klub:** registrira natjecatelje i prijavljuje ih na natjecanje

Temeljna domena

- Klub, Korisnik, Natjecatelj, Natjecanje, Prijava
- Rokovi nominacija: **prelim** i **final**

Pravila rokova - nominacije

Faza	Što klub može
Do prelim roka	Dodavati nove prijave, uređivati podatke prijave, odjaviti prijavu
Između prelim i final	Odjaviti natjecatelja ili promijeniti kategoriju - ne može dodavati nove prijave
Nakon final roka	Nema izmjena

Natjecanje ima dva ključna roka: **preliminarne nominacije** (`prelim_deadline`) i **finalne nominacije** (`final_deadline`).

Zašto je ova tema dobra za učenje?

- Prirodno sadrži **role** i **ownership** - admin vidi sve, klub samo svoje
- Ima **workflow** i vremenska poslovna pravila - realan backend, ne samo CRUD
- Relacije i constrainti - realan rad s bazom
- Frontend je zanimljiviji: tablice, filteri, statusi, onemogućeni gumbi po fazi
- Obrana po kodu je jasna: može se postaviti **precizno pitanje** i provjeriti razumijevanje

Tehnologije koje koristimo

Backend

FastAPI + Uvicorn | Pydantic | SQLAlchemy 2.0 | Alembic | PostgreSQL

Frontend

Vue 3 + Vite + TypeScript | Vue Router | Pinia | Axios

Ops / workflow

Docker Compose | Git | uv | pytest | ruff | CI | Deploy (Railway ili Render)

Arhitektura aplikacije

Frontend (SPA) komunicira s backendom preko **REST API-ja**. Backend implementira:

- autentikaciju (JWT) i autorizaciju (role/ownership)
- poslovna pravila i rad s bazom
- dokumentaciju API-ja (OpenAPI/Swagger)

Baza je **jedini izvor istine** za podatke.

Frontend je „glup” za pravila - backend je autoritet. UI može sakriti opcije, ali backend **mora zabraniti**.

Struktura projekta

Backend

- `routes/` - HTTP rute i parametri
- `schemas/` - Pydantic DTO modeli
- `models/` - SQLAlchemy modeli
- `services/` - poslovna logika
- `repositories/` - DB upiti
- `core/` - config, auth, errors

Frontend

- `src/views/` - stranice
- `src/components/` - UI komponente
- `src/stores/` - Pinia storeovi
- `src/router/` - rute i guardovi
- `src/api/` - API client wrapper

Cilj je izbjeći „sve u jednom file-u”. Svaki sloj ima jednu odgovornost.

Raspored predavanja po temama

Pred.	Datum	Tema
0	05.10.	O predmetu
1	05.10.	Backend arhitektura i skeleton projekta
2	12.10.	Modeliranje podataka i rad s bazom
3	19.10.	Autentikacija i sigurnost (JWT)
4	26.10.	Autorizacija, role i ownership
5	02.11.	API dizajn, validacija i CRUD domene
6	09.11.	Poslovna pravila i workflow u backendu
-	16.11.	Obrana 1. dijela projekta (backend)
7	23.11.	Arhitektura frontend aplikacija (Vue)
8	30.11.	Komunikacija frontend-backend i auth flow
9	07.12.	State management i UX discipline
10	14.12.	Kompleksni UI, tablice i administracija
11	21.12.	Testiranje full-stack aplikacije
	28.12. - 04.01.	<i>Božićni praznici</i>
12	11.01.	Frontend u produkciji i deploy
-	18.01.	Obrana 2. dijela projekta (full-stack)

Projekt studenata - zahtjevi

- Tijekom semestra izrađujete **vlastitu full-stack web aplikaciju**
- Aplikacija mora sadržavati:
 - autentikaciju (JWT) i barem dvije role
 - najmanje dva entiteta i njihove relacije
 - ownership - korisnik vidi i uređuje svoje podatke
 - poslovna pravila: workflow, statusi ili rokovi
 - frontend SPA s osnovnim UX stanjima: loading, error, empty
- **Predaja 1. dijela:** backend funkcionalan, predan na GitHub - 45 bodova
- **Predaja 2. dijela:** full-stack na GitHubu i **deployan** - 45 bodova
- Ako ne napravite sve tijekom nastave, imate još jedan termin predaje za vrijeme ispitnih rokova u veljači

Bodovanje

Dolazak na predavanje nosi **1 bod**, najviše **10 bodova** u semestru.

Komponenta	Bodovi
1. dio projekta (backend)	45
2. dio projekta (frontend + deploy)	45
Dolasci na predavanja	10
Ukupno	100

Predavanja ima 12, a bodovanih dolazaka 10 - možete propustiti dva termina bez posljedica.

Dolazak nije obavezan, ali sada nosi dio ocjene. Najviše ćete ionako dobiti ako dolazite jer želite nešto naučiti.

Primjeri tema projekata

- Sustav rezervacija - termini, dvorane, treneri, konzultacije
- Event management - događaji, prijave, kapacitet, check-in
- Task / workflow sustav - projekti, zadaci, statusi, komentari
- Coach & athlete tracker - programi, treninzi, napredak
- Dokument approval sustav - zahtjevi, odobravanje, verzije
- Mini marketplace - oglasi, poruke, favoriti
- Evidencija članstva - klubovi, članovi, uplate, statusi

Odaberite temu koja ima **uloge i pravila**, a ne samo jednostavan CRUD.

Kako izgleda obrana

Demonstracija funkcionalnosti, pregled repozitorija i **ključnih dijelova koda**, pitanja o odlukama i edge caseovima. Moguća mini promjena uživo: dodaj polje, migraciju, prilagodi DTO/UI.

Tipična pitanja

- Gdje provjeravaš ownership?
- Kako backend zabranjuje izmjene nakon roka?
- Zašto je status code 401/403/409/422?
- Gdje i kako hendlaš 401 na frontendu?

Lokalni setup

- Git
- Python 3.11+ za backend
- Docker Desktop + Docker Compose za bazu
- Node.js 22+ za frontend
- Editor: VS Code / IntelliJ / drugi

Pokretanje projekta lokalno

1) Baza (Postgres)

```
docker compose up -d db
```

2) Backend

```
cd api
```

```
uv venv
```

```
source .venv/bin/activate
```

```
uv pip install -r requirements.txt
```

```
alembic upgrade head
```

```
python -m app.seed
```

```
uvicorn app.main:app --reload
```

3) Frontend

```
cd ../web
```

```
npm install
```

```
npm run dev
```

Kvaliteta koda: što je „dovoljno dobro”

Backend

- Validacija inputa (Pydantic)
- Dosljedni errori (standardiziran JSON)
- Logika u service layeru, ne u routeru
- Testovi za ključna pravila

Frontend

- Loading/error/empty stanja
- Route guardovi za role
- Form validation
- Bez hardkodiranih URL-ova (env)

Testiranje

Test nije „još jedan zadatak” - to je **osiguranje** da pravila vrijede. Fokus testova u ovom projektu:

- Autentikacija - login/refresh
- Autorizacija - role/ownership
- Rokovi - prije prelim / između / nakon final
- Unikatan constraint - nema duple prijave

Na obrani ću vas pitati **što točno test pokriva i zašto je to bitno.**

Deploy: što znači „radi u produkciji”

Deploy nije samo „imam URL”. Morate razumjeti:

- Env varovi i tajne - JWT secret, DB URL
- CORS - frontend domena ↔ backend
- Migracije u produkciji - alembic upgrade
- Error logging - što logiramo, što ne

Koristit ćemo moderni PaaS - Railway ili Render.

Akademski čestitost i AI alati

AI možete koristiti za pomoć, ali na obrani morate **razumjeti i objasniti svoj kod**. Kopirani kod bez razumijevanja se vidi odmah na usmenom.

Ako koristite vanjske snippete, morate znati:

- Gdje su rizici za sigurnost
- Što se događa u edge caseovima
- Kako to testirate

Kako učiti ovaj kolegij

- Svako predavanje: pogledaj diffove, reproduciraj korake u svom repou
- Ne gomilaj dugačke sesije - radi **malo i često**
- Debugging rutina:
 - pročitaj error, izoliraj uzrok, napravi najmanji repro
 - provjeri request/response - status code, payload
 - koristi logove i devtools
- Kad zapne: pokaži **konkretan problem** - log, request, commit